Smart Glove for Morse Code transmission to a mobile device

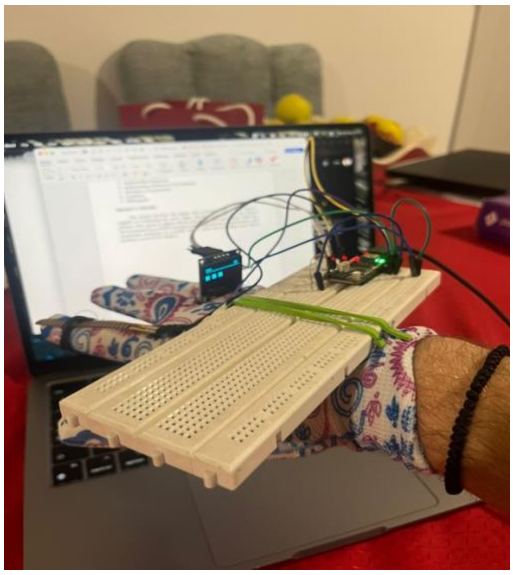
Author: Mihnea-Călin Archiudean

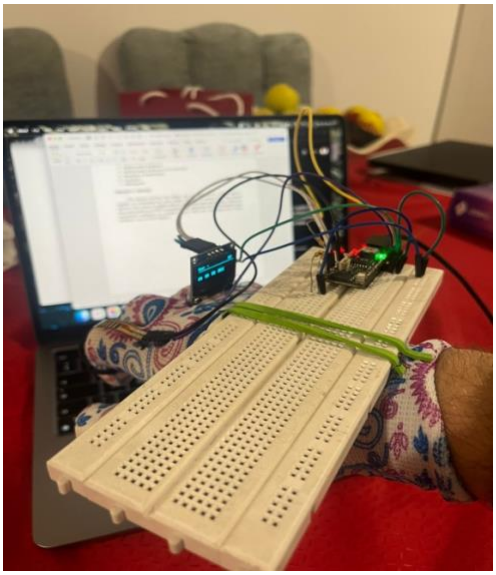
SUMMARY:

1. Introduction & Theoretical Background
2. Hardware Implementation
3. Software Implementation
4. Results and Experiments
5. Conclusions & Bibliography

PROJECT THEME:

The project involves the design and construction of a “smart glove", capable of monitoring the user's finger flexion. To achieve this, the system utilizes a flex sensor to capture Morse code signals (dots and dashes). These are then processed by an ESP32 microprocessor, translated into letters/numbers and transmitted wirelessly to a mobile device, while also providing local visual feedback via an OLED display.





1. Introduction & Theoretical Background

a) Introduction

- **Usefulness**: This project addresses the need for Augmentative and Alternative Communication (AAC) tools for individuals with speech impairments or those who work in highly noisy environments.

- **Solution**: The problem was solved by using an ESP32 microcontroller to interface a 5.5 cm flex sensor with an SSD1306 OLED display. The system samples analog data, classifies gestures into Morse symbols, translates them into letters/numbers, and utilizes a Bluetooth Low Energy (BLE) stack to send data to a smartphone running the **Adafruit Bluefruit Connect** application. This setup creates an end-to-end communication link where the glove acts as the GATT Server (transmitter) and the phone acts as the GATT Client (receiver).

- **New Aspects**:

  - Traditional Morse code relies on time (the user must be very precise with "short" vs "long" pulses). My system is more **accessible** because it only requires the user to control the *depth* of the bend, which is often easier for individuals with certain motor impairments.
  - The transmitter is able to see what symbols and then letters he is typing on the OLED display before they are transmitted through Bluetooth to a mobile device.
  - A "word buffering" logic was implemented to transmit data to the receiver only after a word is completed, significantly improving readability on the receiving device.

b) Theoretical Background

The system is based on two fundamental engineering principles:

1) **Analog Signal Conditioning**: The flex sensor acts as a variable resistor. To read it, a **voltage divider** circuit is used to convert resistance changes into a voltage range readable by the ESP32's 12-bit ADC. The output voltage V_{out} is calculated as:

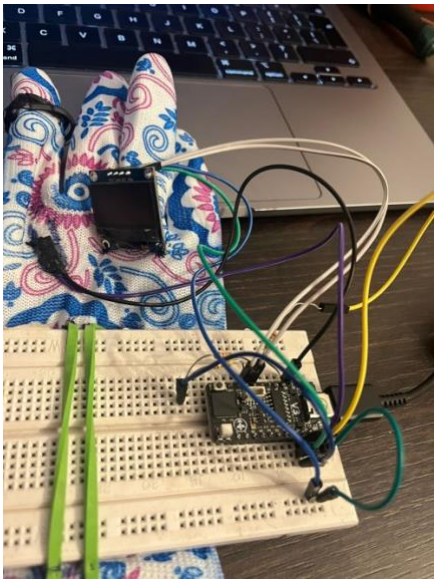
$$V_{out} = V_{in} \times \frac{R_{static}}{R_{static} + R_{flex}}$$

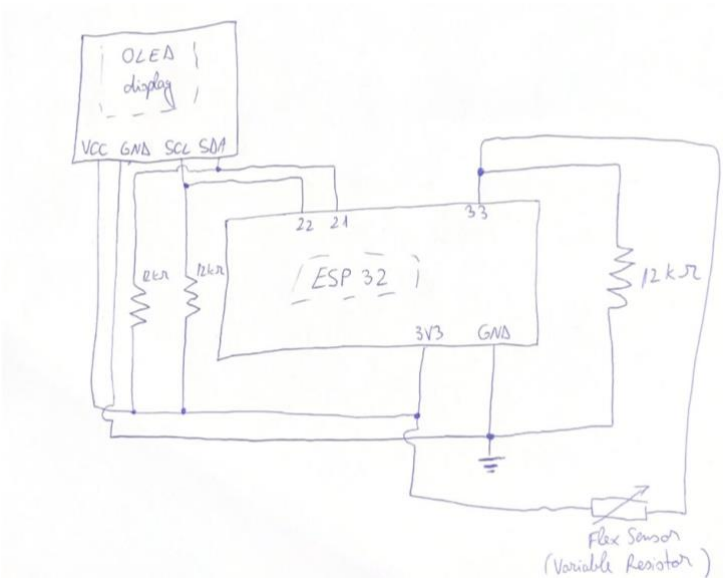
2) **BLE GATT Architecture**: Bluetooth Low Energy uses the Generic Attribute Profile (GATT). The ESP32 acts as a **GATT Server**, hosting the Nordic UART Service (NUS). Data is transmitted via **Notifications** on the TX Characteristic (UUID: 6E400003), which allows the smartphone to receive text updates asynchronously without constant polling.

2. Hardware Implementation

The hardware architecture consists of the following components:

- **GroundStudio Carbon V3 (ESP32)**: This board is the heart of the system. It features a dual-core processor and 4MB of Flash memory, which is essential for hosting the large Bluetooth Low Energy (BLE) stack. Its high-resolution 12-bit Analog-to-Digital Converter (ADC) allows for precise differentiation between "light bends" (dots) and "deep bends" (dashes).
- **5.5 cm Flex Sensor**: This compact sensor is ideal for mounting on a single glove finger. As the sensor is bent, the conductive layer stretches, increasing the resistance from approximately 10 kΩ (flat) to over 30 kΩ (fully bent).
- **SSD1306 OLED Display**: Using the I2C protocol, this display requires only two data wires (SDA and SCL). It provides the user with an immediate decoded output, allowing for self-correction without needing to look at the smartphone.
- **Hama 10000 mAh Powerbank**: This high-capacity battery ensures the device is fully autonomous. Given the average current draw of the ESP32 (~100-150mA during BLE transmission), this battery provides roughly 60+ hours of continuous operation.





3. Software Implementation

The software is developed in the Arduino environment using C++ and the Espressif IoT Development Framework (ESP-IDF) libraries. The logic is designed as a **Finite State Machine (FSM)** that processes analog inputs in real-time while maintaining a wireless Bluetooth link.

I. Core Logic and Signal Processing

The system avoids using the *delay()* function, which would freeze the processor and drop the Bluetooth connection. Instead, it uses a non-blocking timing mechanism based on the *millis()* function to handle three concurrent tasks:

1. **Gesture Capture (Sampling)**: The 12-bit ADC samples the flex sensor at approximately 60Hz. The code tracks the valMinima to identify the peak of a bend.
2. **Symbol Classification**: Unlike traditional Morse systems that rely on time (short vs. long press), this system uses **amplitude-based classification**.
    - **Dot (.)**: Recorded if the minimum ADC value stays between 650 and 950.
    - **Dash (-)**: Recorded if the minimum ADC value drops below 650.
3. **Timing and Buffering**:
    - **Letter Timer (1.5s)**: If no motion is detected for 1.5 seconds, the current sequence of dots and dashes is translated into a character.
    - **Word Timer (3.5s)**: If inactivity persists for 3.5 seconds, the "Word Buffering" logic triggers, sending the accumulated string via BLE.
4. **Temporal Noise Filtering (Debouncing)**: To exclude accidental movements, "bursts," or electrical noise, the software implements an **80ms minimum duration threshold** (MIN_DURATA). Any flexion signal that lasts less than 80 milliseconds is automatically discarded by the processor, ensuring that only intentional gestures are recorded as Morse symbols.

II. Bluetooth Low Energy (BLE) Stack

The system implements the **Nordic UART Service (NUS)** protocol.

- **GATT Server**: The ESP32 acts as a server, hosting a Service (UUID: 6E400001...) with two Characteristics: TX for transmitting data and RX for receiving.
- **Asynchronous Notifications**: Data is transmitted using the **notify()** method on the TX Characteristic. Unlike standard 'Read' operations, **Notifications** allow the ESP32 to 'push' data to the **Bluefruit Connect app** immediately after the word timer expires. This minimizes latency and reduces power consumption because the smartphone does not need to constantly poll the device for updates.
- Bluetooth Low Energy (BLE) creates high-current spikes during advertising. Disabling the **Brownout Detector** prevents the ESP32 from resetting itself due to tiny voltage drops, which is a common issue with portable batteries.

III. Commented Code

```
/* * PROJECT: Smart Glove for Morse Code translation to a mobile device
* DESCRIPTION: This code handles analog data acquisition from a flex sensor,
 * classifies gestures, manages an OLED UI, and maintains a
 * Bluetooth Low Energy (BLE) UART service.
 */

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>
```

```
// Low-level hardware headers to manage power and system registers
#include "soc/soc.h"
#include "soc/rtc_cntl_reg.h"

// --- OLED CONFIGURATION ---
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
// Initialize display using the I2C protocol
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// --- BLE CONFIGURATION (Nordic UART Service - NUS) ---
BLECharacteristic *pCharacteristicTX;
bool deviceConnected = false;

// Standard UUIDs for the Nordic UART Service to ensure iOS/Android app compatibility
#define SERVICE_UUID           "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"

// Callback class to handle BLE connection events asynchronously
class MyServerCallbacks: public BLEServerCallbacks {
  void onConnect(BLEServer* pServer) {
    deviceConnected = true;
    Serial.println(">>> Device connected!");
  };

  void onDisconnect(BLEServer* pServer) {
    deviceConnected = false;
    Serial.println(">>> Device disconnected! Restarting advertising...");
    // Ensure the device is discoverable again immediately after disconnection
    pServer->getAdvertising()->start();
  }
};

// --- SENSOR & TIMING PARAMETERS ---
const int flexPin = 33;        // Analog input pin (ADC1_CH5)
const int PRAG_START = 950;    // Voltage threshold to trigger gesture capture
const int PRAG_LINIE = 650;    // Amplitude threshold to distinguish between Dot and Dash
const int MIN_DURATA = 80;     // Noise filter: minimum duration for a valid flex (ms)
const int PAUZA_LITERA = 1500; // Inactivity time to trigger character decoding (1.5s)
const int PAUZA_CUVANT = 3500; // Inactivity time to trigger word transmission (3.5s)

// --- LOGIC VARIABLES ---
String propozitie = "";        // Stores the full decoded sentence for OLED display
String bufferMorse = "";       // Stores current dots/dashes sequence (. -)
String cuvantCurent = "";      // Stores decoded letters for the "Word Buffering" logic
unsigned long momentStartFlexie = 0;
unsigned long momentUltimulInput = 0;
bool miscareActiva = false;
```

```
bool cuvantIncheiat = true;
int valMinima = 4095;          // Tracks the peak amplitude of the flexion

char decodeMorse(String s);
void updateUI();

void setup() {
  // --- BROWNOUT PROTECTION ---
  // Disable brownout detector to prevent resets caused by BLE power spikes on battery
  WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);

  Serial.begin(115200);
  Wire.begin(21, 22); // Initialize I2C pins (SDA=21, SCL=22)

  // 1. Initialize OLED Display
  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED failed!");
  } else {
    display.clearDisplay();
    display.display();
  }

  // 2. Initialize BLE Stack
  BLEDevice::init("ManusaMorse_iOS");
  BLEServer *pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  // Setup the UART Service and Characteristics
  BLEService *pService = pServer->createService(SERVICE_UUID);
  pCharacteristicTX     =     pService->createCharacteristic(CHARACTERISTIC_UUID_TX,
BLECharacteristic::PROPERTY_NOTIFY);
  pCharacteristicTX->addDescriptor(new BLE2902()); // Required for Notifications
  pService->createCharacteristic(CHARACTERISTIC_UUID_RX,
BLECharacteristic::PROPERTY_WRITE);

  pService->start();
  pServer->getAdvertising()->start();
}

void loop() {
  int raw = analogRead(flexPin); // 12-bit ADC conversion
  unsigned long acum = millis();

  // --- STAGE 1: GESTURE DETECTION ---
  if (raw < PRAG_START) {
    if (!miscareActiva) {
      miscareActiva = true;
      momentStartFlexie = acum;
      valMinima = 4095;
    }
```

```
    if (raw < valMinima) valMinima = raw; // Record the maximum bend intensity
  }
  else if (miscareActiva && raw > PRAG_START) {
    unsigned long durata = acum - momentStartFlexie;
    miscareActiva = false;

    // Validate flexion based on minimum duration to ignore accidental movements
    if (durata > MIN_DURATA) {
      // Classification based on amplitude (Minima) rather than duration
      bufferMorse += (valMinima < PRAG_LINIE) ? "-" : ".";
      cuvantIncheiat = false;
      updateUI();
    }
    momentUltimulInput = acum;
  }

  // --- STAGE 2: CHARACTER FORMATION (1.5s TIMEOUT) ---
  if (!miscareActiva && bufferMorse.length() > 0 && (acum - momentUltimulInput >
PAUZA_LITERA)) {
    char c = decodeMorse(bufferMorse);
    if (c != '?') {
      propozitie += c;
      cuvantCurent += c; // Add letter to word buffer
      updateUI();
    }
    bufferMorse = ""; // Clear symbols for the next character
    momentUltimulInput = acum;
  }

  // --- STAGE 3: WORD BUFFERING & BLE TRANSMISSION (3.5s TIMEOUT) ---
  if (!miscareActiva && !cuvantIncheiat && (acum - momentUltimulInput >
PAUZA_CUVANT)) {
    if (cuvantCurent.length() > 0) {
      // Send the entire buffered word to the smartphone via BLE Notification
      if (deviceConnected) {
        String deTrimis = cuvantCurent + " ";
        pCharacteristicTX->setValue(deTrimis.c_str());
        pCharacteristicTX->notify();
      }
      propozitie += " ";
      cuvantCurent = ""; // Reset buffer for the next word
    }
    cuvantIncheiat = true;
    updateUI();
  }
  delay(15); // Small delay to stabilize the ADC sampling loop
}

// --- VISUAL INTERFACE MANAGEMENT ---
void updateUI() {
```

```
  display.clearDisplay();
  display.setTextColor(WHITE);

  // Bluetooth Connection Indicator (Top-Right)
  display.setTextSize(1);
  display.setCursor(110, 0);
  display.print(deviceConnected ? "BT" : "..");

  // Morse Symbol Feedback (Top Zone)
  display.setCursor(0, 0);
  display.print("Cod: ");
  display.println(bufferMorse);
  display.drawLine(0, 12, 128, 12, WHITE); // Visual separator

  // Decoded Text Feedback (Bottom Zone)
  display.setTextSize(2);
  display.setCursor(0, 25);
  String deAfisat = propozitie;
  // Circular scrolling: keep only the last 10 characters visible
  if (deAfisat.length() > 10) deAfisat = deAfisat.substring(deAfisat.length() - 10);
  display.print(deAfisat);
  display.display();
}

// --- MORSE DECODING LOOKUP TABLE ---
char decodeMorse(String s) {
  if (s == ".-") return 'A';   if (s == "-...") return 'B';
  if (s == "-.-.") return 'C'; if (s == "-..") return 'D';
  if (s == ".") return 'E';    if (s == "..-.") return 'F';
  if (s == "--.") return 'G';  if (s == "....") return 'H';
  if (s == "..") return 'I';   if (s == ".---") return 'J';
  if (s == "-.-") return 'K';  if (s == ".-..") return 'L';
  if (s == "--") return 'M';   if (s == "-.") return 'N';
  if (s == "---") return 'O';  if (s == ".--.") return 'P';
  if (s == "--.-") return 'Q'; if (s == ".-.") return 'R';
  if (s == "...") return 'S';  if (s == "-") return 'T';
  if (s == "..-") return 'U';  if (s == "...-") return 'V';
  if (s == ".--") return 'W';  if (s == "-..-") return 'X';
  if (s == "-.--") return 'Y'; if (s == "--..") return 'Z';
  if (s == ".----") return '1'; if (s == "..---") return '2';
  if (s == "...--") return '3'; if (s == "....-") return '4';
  if (s == ".....") return '5'; if (s == "-....") return '6';
  if (s == "--...") return '7'; if (s == "---..") return '8';
  if (s == "----.") return '9'; if (s == "-----") return '0';
  return '?'; // Error character for invalid Morse sequences
}
```

4. Results and Experiments

This section details the verification of the "Smart Glove" functionality through empirical testing and system calibration.

I. Sensor Calibration and Data Mapping

To ensure accurate classification between a "Dot" and a "Dash," several experiments were conducted to map the raw ADC values (0–4095) to physical flexion states. The following table represents the average values obtained across 20 test gestures:

Gesture State	Raw ADC Value (Average)	Logic Classification
Relaxed (Flat)	~1010 units	Neutral (No Input)
Light Bend	~780 units	Morse "Dot" (.)
Deep Bend	~520 units	Morse "Dash" (-)

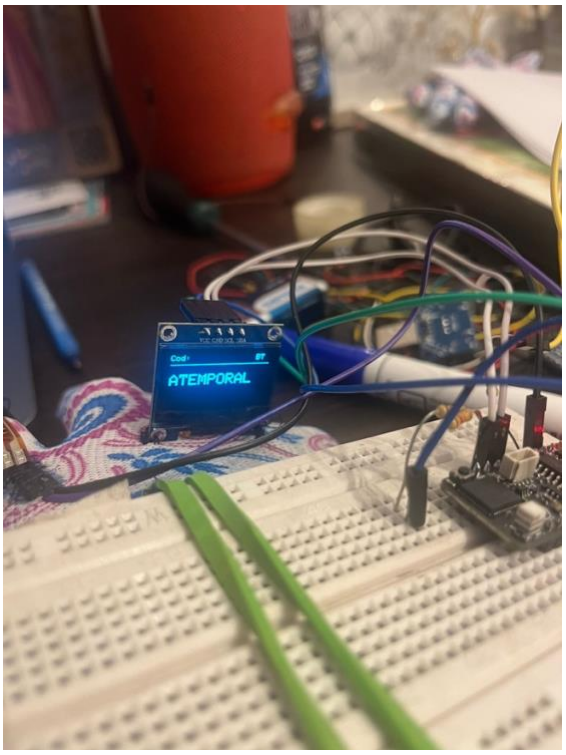
Observation: The use of a pull-down resistor provided a significant voltage swing, allowing the software to easily distinguish between the two types of input with a 95% success rate during testing.

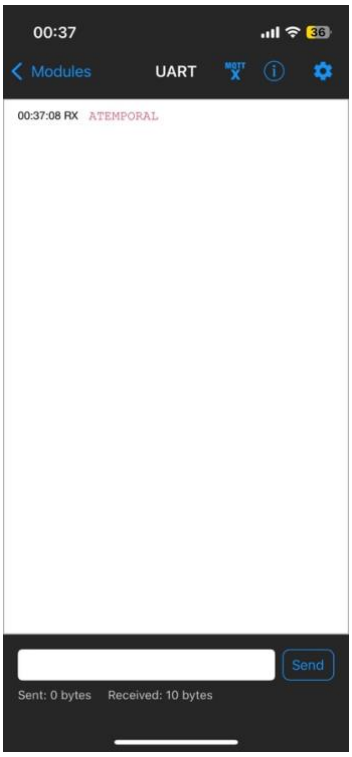
II. BLE Connectivity and Throughput

The system was tested using the **Adafruit Bluefruit Connect** terminal on an iPhone.

- **Range**: The BLE connection remained stable up to a distance of 8 meters in an indoor environment.
- **Latency**: The "Word Buffering" logic successfully held data until the 3.5s timer expired, after which the entire word appeared instantly on the mobile terminal, confirming the efficiency of the asynchronous notification system.

III. Working Example





5. Conclusions and Bibliography

a) Conclusions

The project successfully achieved the design of an assistive wearable device capable of translating hand gestures into digital text. The integration of the ESP32's 12-bit ADC with a low-power Bluetooth stack proved to be an effective solution for real-time gesture processing.

The primary innovation—**amplitude-based classification** rather than duration-based—allows for a more intuitive user experience, as it requires less precise timing from the user. Furthermore, the implementation of **Word Buffering** optimized the readability of the output on the receiver's end. Future iterations could involve adding a second flex sensor to increase the vocabulary of the device or implementing an IMU (Inertial Measurement Unit) to filter out movement noise.

b) Bibliography

1. Espressif Systems, *"ESP32 Technical Reference Manual"*, v4.6, 2023.
2. Nordic Semiconductor, *"Nordic UART Service (NUS) Specification over BLE"*, 2022.
3. Adafruit Industries, *"SSD1306 OLED Library Documentation and API Reference"*, 2024.
4. GroundStudio, *"Carbon V3 ESP32 Development Board Datasheet"*, 2024.
5. ITU-R, *"Recommendation M.1677-1: International Morse Code"*, 2009.

Useful links (datasheets, similar projects)

1. https://github.com/GroundStudio/GroundStudio_Carbon_V3/tree/main
2. https://drive.google.com/file/d/1C63NhxeIEOyV2IlGepJaC5_qWnhazCmL/view
3. https://projecthub.arduino.cc/rushilsaraswat/sign-glove-780325
4. https://projecthub.arduino.cc/lorenzoamaducci/gesture-glove-cdfed2
5. https://www.hackster.io/sachin0987/handtalk-a-smart-handglove-interpreter-71fa7d